

Introduction to

API Testing

From Manual Testing to the API Layer — Concepts, Tools & Hands-On Practice

REST · Postman · Newman · Swagger · Authentication · Best Practices

What We'll Cover Today

01

What is an API?

Core concept & anatomy of API communication

03

Why & When API Testing?

Shift-left, integration gates & the testing pyramid

05

Swagger & Endpoints

Reading API docs & interpreting endpoint structures

07

Bugs in API Testing

Types of defects unique to the API layer

09

Microservices & Contract Testing

API testing in distributed architectures

11

Business Use Case

End-to-end scenario: automate these APIs!

02

Types of APIs

REST, SOAP, GraphQL, gRPC & when to use each

04

HTTP Methods

GET, POST, PUT, PATCH, DELETE – what to verify

06

Authentication Methods

Basic, Bearer, API Key, OAuth 2.0

08

Tools - Postman & Newman

Demo-ready tooling for REST API testing

10

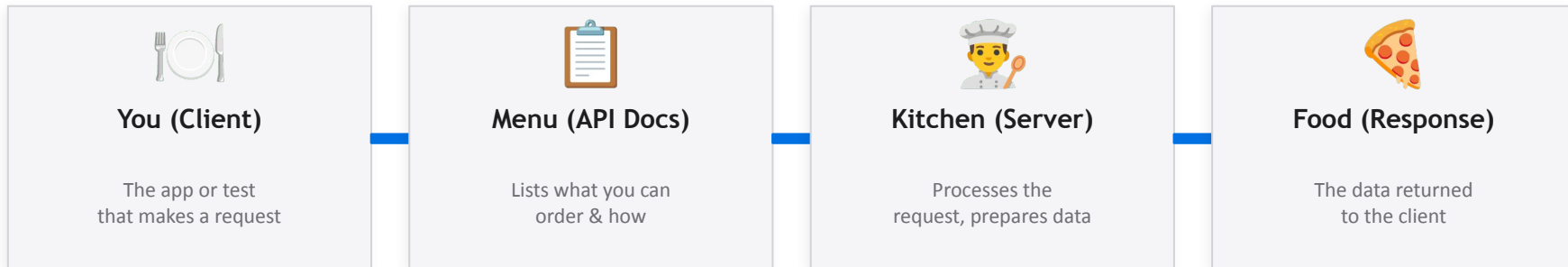
Best Practices

10 habits of effective API testers

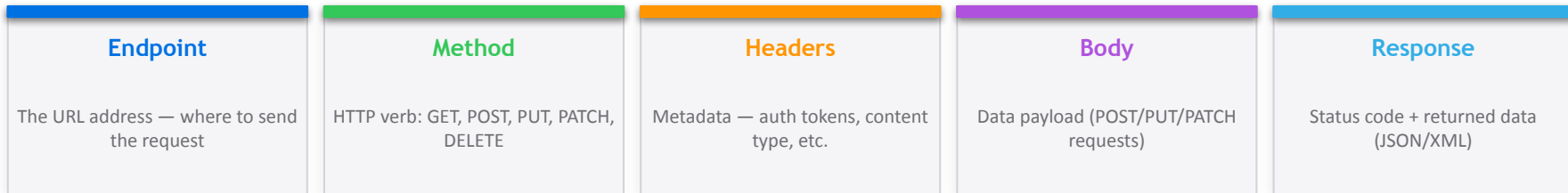
What is an API?

API = Application Programming Interface

A contract that allows two software systems to communicate — defining what requests can be made, how to make them, and what responses to expect.



Key Components of an API Call



Types of APIs

REST

Representational State Transfer

- › Uses HTTP methods (GET, POST...)
- › Stateless – each request independent
- › Returns JSON or XML
- › Most widely used today

📌 Use when: Web & mobile apps, microservices

★ Focus of this session

SOAP

Simple Object Access Protocol

- › XML-based messaging
- › Strict standards & contracts
- › WS-Security built-in
- › Common in enterprise/banking

📌 Use when: Legacy enterprise systems, banking

Not in scope today

GraphQL

Query Language for APIs

- › Client specifies exact data needed
- › Single endpoint for all queries
- › Reduces over/under-fetching
- › Used by Facebook, GitHub

📌 Use when: Complex UIs with flexible data needs

Advanced topic

gRPC

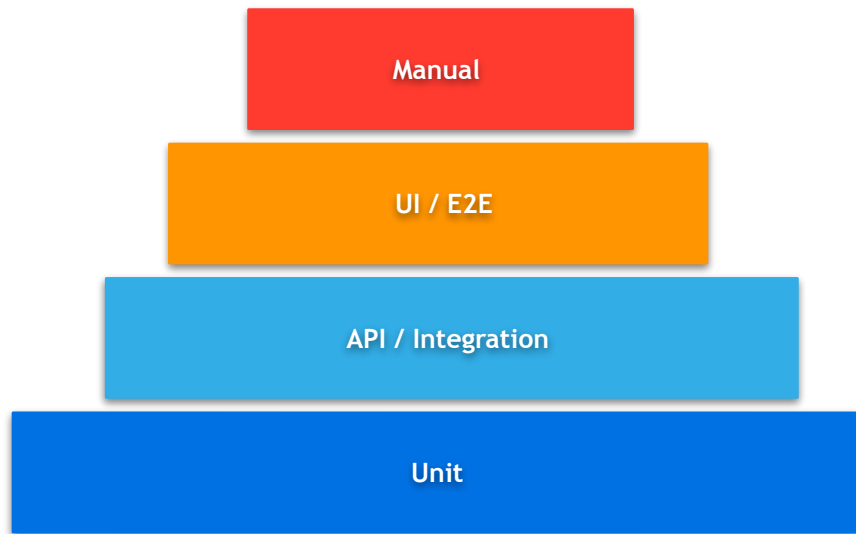
Google Remote Procedure Call

- › Uses Protocol Buffers (binary)
- › Very high performance
- › Strongly typed contracts
- › Used in microservices

📌 Use when: Internal microservice-to-service comms

Advanced topic

Why & When Do We Do API Testing?



Testing Pyramid

Why Test APIs?



Faster Feedback

APIs tested before UI is built — bugs found early are 10× cheaper to fix



Integration Gate

Validates two systems (your app + 3rd party) communicate correctly



Cost-Efficient

No browser needed; runs in seconds vs minutes for UI tests



Security Layer

Catch exposed endpoints, auth bypass, and data leaks early



Shift-Left

Run in CI/CD pipelines — every code commit is validated automatically

Reading Swagger / OpenAPI Documentation

Swagger is a visual, interactive API documentation tool based on the OpenAPI Specification. It lets you read, understand, and try out API calls directly in the browser.

ShopEasy API v1.0 | /swagger-ui

GET /orders/{id} — Get order by ID

POST /orders — Create new order

PUT /orders/{id}/cancel — Cancel order

DEL /cart/items/{itemId} — Remove from cart

Responses: 200 ✓ 201 ✓ 400 X 401 X 404 X 500 X

Schema: { "orderId": string, "status": string, "total": number }

Key Things to Look For in Swagger:

1

Endpoint URL & HTTP Method

Note the exact path, method verb, and path parameters like {id}

2

Request Parameters

Path params, query params (?key=val), and request body schema

3

Authentication Required

Look for 🔒 lock icon — Bearer/OAuth needed for protected endpoints

4

Response Schema

Understand the JSON structure for your test assertions

5

Response Status Codes

Document all: 200, 201, 400, 401, 404, 409, 500

6

Try It Out!

Execute calls live — great for initial exploration before scripting

Interpreting API Endpoints

`https:// api.shopeasy.com /v1 /orders /1042 ?status=open`

Protocol

Host

Version

Resource

Path Param

Query Param

Common Endpoint Patterns

GET

`/api/users`

List all users (collection endpoint)

GET

`/api/users/{id}`

Get specific user by ID

PUT

`/api/users/{id}`

Update specific fields only

GET

`/api/users/{id}/orders`

Nested resource: user's orders

POST

`/api/users`

Create a new user

PUT

`/api/users/{id}`

Replace user entirely (all fields)

DEL

`/api/users/{id}`

Delete the user

GET

`/api/products?category=shoes&sort=price`

Filter + sort via query params

HTTP Methods – What to Verify

GET

Retrieve data

Verify:

- › Status 200 with expected data
- › Response schema matches contract
- › Pagination parameters work
- › Empty/null handled gracefully
- › Unauthorized returns 401

POST

Create a new resource

Verify:

- › Status 201 + Location header
- › Resource created in DB
- › Duplicate entry returns 409
- › Missing required fields → 400
- › Response body has generated ID

PUT

Replace entire resource

Verify:

- › Status 200 or 204 on success
- › All fields updated (not partial)
- › Non-existent resource → 404
- › Idempotent: same call = same result
- › Validation on required fields

PATCH

Partial update of resource

Verify:

- › Only sent fields are updated
- › Other fields remain unchanged
- › Status 200 with updated body
- › Invalid field → 400 Bad Request
- › Concurrent patch conflicts → 409

DELETE

Remove a resource

Verify:

- › Status 204 (No Content)
- › Resource is gone (GET → 404)
- › Already deleted → 404 or 204
- › Cascade deletes if applicable
- › Only authorized user can delete

API Authentication Methods

API Key

How it works:

A static token passed in the header or query string

```
Header: x-api-key: abc123xyz
Query: ?api_key=abc123
```

✓ Simple to implement, no expiry by default

⚠ Risky if leaked, no user context

Used for:

Internal tools, public APIs

In Postman: Authorization tab → select type

Basic Auth

How it works:

Base64-encoded username:password in Authorization header

```
Authorization: Basic
dXNlcjpwYXNz (base64)
```

✓ Universal support, easy to use

⚠ Sends credentials in every request

Used for:

Legacy systems, dev/test envs

In Postman: Authorization tab → select type

Bearer / JWT

How it works:

A signed token obtained after login, sent in every request header

```
Authorization: Bearer
eyJhbGciOiJIUzI1...
```

✓ Stateless, carries user info, expirable

⚠ Token must be refreshed; handle expiry

Used for:

Most modern REST APIs, SPAs

In Postman: Authorization tab → select type

OAuth 2.0

How it works:

Delegated access — user authorizes app to act on their behalf

```
Step 1: Get auth code
Step 2: Exchange for token
```

✓ Granular scopes, no password sharing

⚠ Complex flow, multiple redirect steps

Used for:

Social login, 3rd-party integrations

In Postman: Authorization tab → select type

Types of Bugs Detected by API Testing



Functional

- Wrong response data / calculations
- Resource not created after POST
- Deleted resource still returned
- Incorrect business logic in response



Data Validation

- Missing required field validation
- Wrong data type accepted
- Boundary values not enforced
- SQL/script injection not sanitized



Security

- Unauthenticated access allowed
- User A can access User B's data
- Expired JWT token accepted
- Sensitive data in response body



Performance

- Response time > SLA threshold
- API crashes under load test
- N+1 queries causing slow response
- Memory leak over repeated calls



Error Handling

- 500 error on invalid input
- Wrong error code (200 vs 404)
- No error message in response
- Stack trace exposed to client



Integration

- 3rd-party timeout not handled
- Wrong format sent to external
- Webhook/callback not triggered
- Rate limit errors not retried

API Testing Tools

Postman

Primary GUI Tool

Industry-leading API client. Send requests, write JS test scripts, create collections, set environments, and run automation with Newman.

- ✓ Collections & Environments
- ✓ JavaScript test assertions
- ✓ Pre-request scripts
- ✓ Mock Servers
- ✓ Newman CLI runner

Best for: Day-to-day exploration & test authoring

Newman

CLI Runner for Postman

Command-line companion to Postman. Runs Postman collections in terminal, CI/CD pipelines, and generates HTML/JSON reports.

- ✓ Runs Postman collections via CLI
- ✓ CI/CD integration (Jenkins, GitHub Actions)
- ✓ HTML & JUnit report generation
- ✓ Data-driven testing with CSV/JSON
- ✓ Environment variable injection

Best for: Automated regression runs & CI pipelines

REST Client (VS Code)

Lightweight HTTP Client

VS Code extension for sending HTTP requests from .http files. Great for keeping requests in version control alongside your code.

- ✓ Requests as plain .http files
- ✓ Version controlled with source code
- ✓ Environment variables in settings.json
- ✓ Response viewer inline
- ✓ No heavy UI overhead

Best for: Quick API calls alongside dev code

cURL

CLI HTTP Tool

Universal command-line HTTP tool available on every OS. Ideal for scripting, debugging, and quick one-off API calls from terminal.

- ✓ Available everywhere by default
- ✓ Easy to embed in bash scripts
- ✓ Full header & body control
- ✓ Supports all auth types
- ✓ Verbose mode: full request/response

Best for: Debugging & shell scripting

Postman — Your API Testing Workbench

Postman | ShopEasy | Order API Tests

POST

https://api.shopeasy-test.com/v1/orders

Send

Params

Auth

Headers

Body

Tests

```
pm.test("Status is 201", function() {  
  pm.response.to.have.status(201);  
});  
  
pm.test("orderId present", function() {  
  const json = pm.response.json();  
  pm.expect(json.orderId).to.be.a("string");  
});
```

✓

 Status is 201

✓

 orderId present 2 passing · 0 failing

Response | 201 Created | 287ms | JSON

```
{  
  "orderId": "ORD-20240412-9872",  
  "status": "created",  
  "total": 1299.00  
}
```

What to Master in Postman:

Collections

Group related requests by feature or module — run all at once

Environments

Variables for base URL, tokens, IDs. Switch Dev/QA/Prod instantly

Pre-request Scripts

JS that runs before sending: auto-generate tokens, timestamps, dynamic data

Test Scripts

pm.test() assertions: status codes, JSON fields, response time, schema

Collection Runner

Run all requests in sequence. See pass/fail summary. Export results

Newman CLI

newman run collection.json -e env.json — runs in CI/CD pipelines

API Testing Best Practices

01

Test Positive & Negative Paths

Always test the happy path AND invalid inputs, missing fields, wrong types, and boundary values.

03

Use Environment Variables

Never hardcode URLs, tokens, or IDs. Use Postman environments to switch Dev/QA/Prod instantly.

05

Test All Status Codes

Cover 200, 201, 400, 401, 403, 404, 409, 422, 500. Negative tests are as important as positive.

07

Chain Requests Intelligently

Extract values from response (e.g. created ID) and pass to next request using `pm.variables`.

09

Document Your Bug Reports

Log bugs with: method, endpoint, payload, expected response, actual response, and status code.

02

Validate the Full Response

Check status code, response body schema, data types, required fields, AND response headers.

04

Keep Tests Independent

Each test should set up its own data and clean up after. Don't rely on execution order.

06

Verify Response Headers

Check Content-Type, CORS, Cache-Control, and security headers like X-Frame-Options.

08

Automate in CI/CD

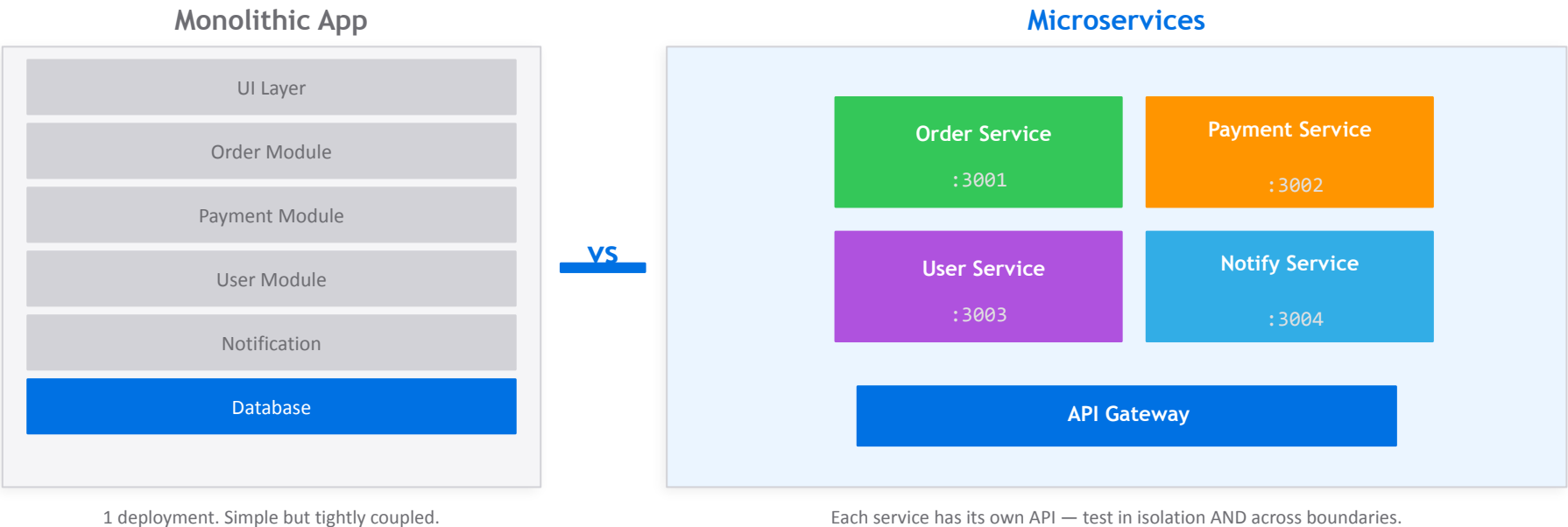
Run Newman in your pipeline so every commit auto-validates the API layer before UI tests.

10

Version Control Your Collections

Export Postman collections as JSON and commit them to the repository alongside the source code.

Microservices & API Testing



Microservices vs Monolith — Impact on API Testing:

	Monolith	Microservices
Scope	Whole application tested together	Each service tested in isolation + integrated
Test Focus	One endpoint calls DB directly	Service-to-service API contracts critical
Collections	Single Postman collection	Collection per service; run via Newman in CI
Mocking	Rarely needed	Mock downstream services with Postman Mocks

Contract Testing vs API Testing

API Testing

What it is:

Testing an API's behavior end-to-end — verifying correct data, status codes, response times, and error handling. You define the expected behavior in test scripts.

- ✓ POST /orders → 201 with orderId
- ✓ GET /orders/999 → 404 Not Found
- ✓ POST /orders (no body) → 400
- ✓ Response time < 500ms

 **Goal: Verify the API works as specified**

Tools: Postman, Newman, REST Assured

Contract Testing

What it is:

A formal agreement (contract) between a consumer (client app) and a provider (API service) defining exactly what request/response structure is expected. Both sides tested against this contract independently.

- ✓ Order Svc expects User Svc → {id, name, email}
- ✓ If User Svc changes schema → contract fails
- ✓ Both services pass without running live
- ✓ Enables parallel team development

 **Goal: Prevent breaking changes between services**

Tools: Pact, Spring Cloud Contract, Postman

VS

 **Real Example: ShopEasy's Order Service (consumer) expects Payment Service (provider) to return { paymentId, status, amount }. Contract test fails if Payment Service renames 'amount' to 'total' — preventing a production outage.**



Business Use Case

ShopEasy — E-Commerce Order Management API

Scenario: You are a QA Engineer for ShopEasy, a growing e-commerce platform. The backend team has deployed a new Order Management microservice. Your task is to validate all APIs before this feature goes live. You have access to the Swagger docs and a test environment.

API Modules to Test:



User Auth



Products



Cart



Orders



Payments



Notifications

Base URL: <https://api.shopeasy-test.com/v1>

Swagger UI: <https://api.shopeasy-test.com/swagger-ui> | Auth: Bearer Token (obtained via POST /auth/login)

Next slide → 12 APIs to automate →

ShopEasy – API Automation Challenge

Method	Endpoint	What to Test / Key Assertions	Module
POST	/auth/login	Get Bearer token; verify 200 + token in response body	Auth
POST	/auth/register	Register user; 201 + userId; duplicate email → 409	Auth
GET	/products	List products; pagination works; filter by category	Product
GET	/products/{id}	Get product by ID; invalid ID → 404	Product
POST	/cart/items	Add item to cart; cart total updates correctly	Cart
GET	/cart	View cart; verify items, quantities, subtotal	Cart
DEL	/cart/items/{itemId}	Remove item; total decreases; non-existent → 404	Cart
POST	/orders	Place order; 201 + orderId; empty cart → 400	Order
GET	/orders/{orderId}	Get order details; verify status, items, total	Order
DEL	/orders/{orderId}/cancel	Cancel order; status=cancelled; shipped → 422	Order
POST	/payments	Initiate payment; 201 + paymentId; duplicate → 409	Payment
GET	/payments/{paymentId}	Verify payment status; invalid ID → 404	Payment

🔗 Challenge: Build a Postman collection for all 12 APIs. Include 1 positive + 1 negative test per endpoint. Run via Newman and share the HTML report. Bonus: Auto-set Bearer token in a

Key Takeaways



APIs are the backbone

of modern software — every app communicates via APIs. Testing them is non-negotiable.



Postman is your primary tool

for exploration, manual testing, and scripted assertions. Newman automates it all.



Always test for security

— auth bypass, unauthorized access, and data leakage are real and common API bugs.



Test earlier, test cheaper

API testing sits below UI in the testing pyramid — faster feedback, lower cost.



Swagger is your best friend

— read it before writing any test. It defines endpoints, schemas, and auth.



Contract testing = team trust

— in microservices, contracts prevent one team's change from breaking another's.

*You now have the foundation. Practice with the ShopEasy challenge, explore Postman deeply, and bring your findings to the next session.
Happy testing! 🚀*